Inverse Kaluza-Klein Scale Symmetry

Document II — Initial Draft

Independent import sections, equation chains, and code

This document is structured as a collection of independent sections, each self-contained. Each section may be read, compiled, or imported separately. Order is thematic, not logically required.

VSEPR-SIM Theoretical Division
Development Day 64

Contents

§1 Framework Identity and Naming

Import block A *standalone, no dependencies*

The Direction of Inference

Standard Kaluza-Klein (KK) logic:

$$\text{extra dimension} \xrightarrow{\text{KK}} \text{force behavior} \tag{A.1}$$

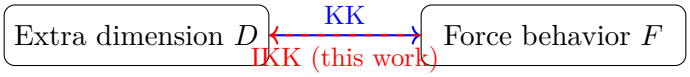
Inverse KK logic (this framework):

$$\boxed{\text{observed force-scale behavior} \xrightarrow{\text{IKK}} \text{implied dimensional permission}} \tag{A.2}$$

The framework does not assume extra dimensions and then derive consequences. It starts from measured force-scale residuals and asks what dimensional permission structure would *produce* those residuals as projections.

Why "Inverse"

The word "inverse" is not metaphorical. The mathematical arrow is literally reversed:



§2 Scale Ladder: Axiom Chain

Import block B *standalone, no dependencies*

Axiom 2.1 (Existence gate)**.** Every physical entity satisfies a binary existence condition:

$$E \in \{0, 1\} \qquad S_0 = (\text{existence}, \ —, \ \lambda_{\text{Planck}}) \tag{B.1}$$

Axiom 2.2 (Scale entry)**.** Each scale level adds one degree of freedom, one mediator, and one observational limit:

$$S_n = (D_n, \ M_n, \ L_n) \tag{B.2}$$

Axiom 2.3 (Packing occurs)**.** Components at scale n self-assemble into identities at scale $n + 1$:

$$\mathbf{C}^{(n)} \xrightarrow{\Pi_n} \mathbf{I}^{(n+1)} \tag{B.3}$$

Packing is irreversible without energy cost $\geq E_{\text{bind}}$.

Axiom 2.4 (Projection)**.** Observation at scale S produces only:

$$\mathcal{O}_S = P_S(\mathcal{I}) \tag{B.4}$$

The full identity $\mathcal{I}$ is not directly accessible.

Axiom 2.5 (Residual existence)**.** The residual is the difference between the full identity content at scale S and what observation recovers:

$$\mathcal{R}_S = \mathcal{U}_S(\mathcal{I}) - \mathcal{E}_S(\mathcal{O}_S) \tag{B.5}$$

$\mathcal{R}_S = 0$ only if projection is perfectly injective at scale S.

§3 Matrix Packing Chain

Import block C *standalone, no dependencies*

C.1 Quark to Particle

Quark component matrix for particle p:

$$\mathbf{C}_q = \begin{bmatrix} q_1 & c_1 & \eta_1 \\ q_2 & c_2 & \eta_2 \\ q_3 & c_3 & \eta_3 \end{bmatrix} \in \mathbb{R}^{3\times 3} \tag{C.1}$$

where columns are charge, color, and helicity channels.

Packing to particle identity row-vector:

$$\mathbf{I}_p = \begin{bmatrix} Q_p & M_p & S_p \end{bmatrix} \in \mathbb{R}^{1\times 3} \tag{C.2}$$

Formal map:

$$\mathbf{I}_p = \Pi_q(\mathbf{C}_q) \qquad 3\times 3 \to 1\times 3 \tag{C.3}$$

C.2 Recursive Packing Ladder

$$\underbrace{\text{quarks}}_{S_2} \xrightarrow{\Pi_2} \underbrace{\text{particles}}_{S_3} \xrightarrow{\Pi_3} \underbrace{\text{atoms}}_{S_3} \xrightarrow{\Pi_3} \underbrace{\text{molecules}}_{S_3} \xrightarrow{\Pi_4} \underbrace{\text{materials}}_{S_4} \xrightarrow{\Pi_5} \underbrace{\text{grav. systems}}_{S_5} \xrightarrow{\Pi_D} \underbrace{\text{dark residuals}}_{S_D} \tag{C.4}$$

C.3 Entropy as Packing Degeneracy

The entropy of a packed identity at scale n:

$$\mathcal{S}_n = k_B \ln \Omega(\mathbf{I}^{(n)}) \tag{C.5}$$

where $\Omega(\mathbf{I}^{(n)})$ counts the number of component configurations $\mathbf{C}^{(n-1)}$ that map to the same $\mathbf{I}^{(n)}$ under Π_{n-1}.

C.4 Complexity

Stable packed identity plus structured residual:

$$\mathcal{C}_n = f\Big(\mathbf{I}^{(n+1)},\ \mathcal{R}_n\Big) \tag{C.6}$$

High complexity requires both a non-trivial packed identity and a non-trivial residual. Pure structure with zero residual is simple. Pure noise with no packed identity is also simple. Complexity is the overlap.

§4 Cancellation and Hidden Intensity

Import block D *standalone, no dependencies*

D.1 The Rule

$$\boxed{\text{net-zero packed value } \neq \text{ no internal structure}} \tag{D.1}$$

D.2 Observable and Hidden Channels

For a component vector $\mathbf{c}_j \in \mathbb{R}^m$:

$$I_j = \boldsymbol{\alpha}^\top \mathbf{c}_j \tag{D.2}$$

$$H_j = \mathbf{c}_j^\top \mathbf{W}\, \mathbf{c}_j \tag{D.3}$$

where $\boldsymbol{\alpha}$ is the projection weight vector and $\mathbf{W}$ is a positive-semi-definite weight matrix for the hidden channel.

Hidden-active condition:

$$I_j = 0, \quad H_j > 0 \tag{D.4}$$

D.3 Neutron-Like Example

Charge projection ($\boldsymbol{\alpha} = [1,1,1]^\top$):

$$I_{\text{charge}} = +\tfrac{2}{3} + (-\tfrac{1}{3}) + (-\tfrac{1}{3}) = 0 \tag{D.5}$$

Diagonal hidden intensity ($\mathbf{W} = \mathbf{I}_{3\times 3}$):

$$H_{\text{charge}} = \left(\tfrac{2}{3}\right)^2 + \left(\tfrac{1}{3}\right)^2 + \left(\tfrac{1}{3}\right)^2 = \tfrac{4}{9} + \tfrac{1}{9} + \tfrac{1}{9} = \tfrac{6}{9} = \tfrac{2}{3} \neq 0 \tag{D.6}$$

---

§5 Dark Matter Kernel Equations

Import block E *standalone, no dependencies*

E.1 Projection Gap Definition

$$\rho_{\text{dark}} = \rho_G - \rho_{\text{EM}} \tag{E.1}$$

$$= P_G(\rho) - P_{\text{EM}}(\rho) \tag{E.2}$$

E.2 Kernel Form

$$\boxed{\rho_{\text{dark}}(x,y,z;\,t) = \int \rho(x,y,z,w;\,t)\Big[K_G(w) - K_{\text{EM}}(w)\Big]\, dw} \tag{E.3}$$

E.3 Kernel Properties (assumed)

Gravitational kernel: couples to all scales

$$\int K_G(w)\, dw = 1 \tag{E.4}$$

EM kernel: truncates above L_{EM}

$$K_{\text{EM}}(w) \approx 0 \quad \text{for } w > L_{\text{EM}} \tag{E.5}$$

The integrand $[K_G(w) - K_{\text{EM}}(w)]$ is therefore non-zero precisely in the dark regime $w > L_{\text{EM}}$.

E.4 Dark fraction

$$f_{\text{dark}} = \frac{\int_{L_{\text{EM}}}^{\infty} \rho(x, y, z, w;\, t)\, K_G(w)\, dw}{\int_0^{\infty} \rho(x, y, z, w;\, t)\, K_G(w)\, dw} \tag{E.6}$$

Observed dark-matter fraction from cosmological measurements: $f_{\text{dark}} \approx 0.27$.

---

§6 Effective Proper Time Chain

Import block F *standalone; references import block B*

F.1 Classical baseline

$$d\tau_{4\text{D}} = dt \sqrt{1 - \frac{v^2}{c^2} + \frac{2\Phi_G}{c^2}} \tag{F.1}$$

Integrated:

$$\tau_{4\text{D}} = \int_0^T dt \sqrt{1 - \frac{v^2(t)}{c^2} + \frac{2\Phi_G(t)}{c^2}} \tag{F.2}$$

F.2 Quantum proper time (ion clock result)

$$\hat{\tau} \equiv \tau(\hat{x}, \hat{p}) \tag{F.3}$$

Proper time becomes quantum-linked. The observable has spread:

$$R_\tau = \hat{\tau} - \langle \hat{\tau} \rangle \tag{F.4}$$

F.3 Dark-scale extension

$$\boxed{d\tau_{\text{eff}} = dt \sqrt{1 - \frac{v^2}{c^2} + \frac{2\Phi_G}{c^2} + \chi_w(w)}} \tag{F.5}$$

Additive decomposition:

$$d\tau_{\text{eff}} = d\tau_{4\text{D}} + d\tau_w \tag{F.6}$$

Limit check:

$$\chi_w(w) \to 0 \implies d\tau_{\text{eff}} \to d\tau_{4\text{D}} \tag{F.7}$$

F.4 Proper-time identity bundle

$$\mathcal{I}_\tau = (\hat{x},\ \hat{p},\ \hat{\tau},\ H_c,\ H_m,\ w) \tag{F.8}$$

Clock-motion relation particle:

$$\mathcal{R}_{cm} = [0,\, 0,\, 0,\, 0 \mid \mathbf{T}_{cm}] \tag{F.9}$$

---

§7 Entanglement: Relation-Particle Chain

Import block G *standalone, no dependencies*

G.1 Relation object

$$\mathcal{R}_{AB} = [\underbrace{0,\ 0,\ 0,\ 0}_{\Delta x,\Delta y,\Delta z,\Delta t} \mid \mathbf{T}_{AB}] \tag{G.1}$$

$$\mathbf{T}_{AB} \in \{-1,\ 0,\ +1\}^m \tag{G.2}$$

G.2 Pair decomposition

$$\mathcal{I}_{AB} \ \rightarrow\ \mathcal{I}_A + \mathcal{I}_B + \mathcal{R}_{AB} \tag{G.3}$$

G.3 Measurement

$$O_A = P_a(\mathcal{I}_A,\ \mathcal{R}_{AB}) \tag{G.4}$$
$$O_B = P_b(\mathcal{I}_B,\ \mathcal{R}_{AB}) \tag{G.5}$$
$$E(a,b) = \langle O_A O_B \rangle \tag{G.6}$$

G.4 No-signal interpretation

$\mathcal{R}_{AB}$ is a fixed structure in identity-space. It does not propagate. Measurement at A reads the relation; measurement at B reads the same relation. No signal travels.

$$\text{real} \neq \text{spatially displaced} \quad , \qquad \text{deterministic} \neq \text{time-evolving signal} \tag{G.7}$$

---

§8 Code: Python Numerical Prototype

Import block H *standalone, Python 3.10+*

The following code implements the core algebraic objects in a minimal self-contained Python prototype. No physics library is required.

```python
import numpy as np
from dataclasses import dataclass, field
from typing import Optional

# --- Scale ladder entry ---------------------------------------------------

@dataclass
class ScaleEntry:
  n:       int
  label:   str
  dof:     str          # degree of freedom name
  mediator: str         # force carrier / mediator name
  L_m:     float        # observational limit (metres)

SCALE_LADDER = [
  ScaleEntry(0, "S0", "existence",   "---",      1.616e-35),
  ScaleEntry(1, "S1", "vector",      "---",      1e-19),
  ScaleEntry(2, "S2", "angular",     "gluon",    1e-15),
  ScaleEntry(3, "S3", "spatial",     "photon",   1e-10),
  ScaleEntry(4, "S4", "time",        "W/Z",      1e-18),
```

```
  ScaleEntry(5, "S5", "curvature",   "graviton", 1e26),
  ScaleEntry(6, "SD", "dark",        "X_D",      float('inf')),
]

# --- Identity component ------------------------------------------------

@dataclass
class Component:
  """One component of an identity bundle at a given scale."""
  scale_n:  int
  value:    np.ndarray   # shape (m,)  -- multi-channel
  weight:   float = 1.0  # alpha_k for anchor computation

# --- Identity bundle ---------------------------------------------------

@dataclass
class IdentityBundle:
  """Full identity state I_i(t)."""
  label:      str
  components: list[Component] = field(default_factory=list)

  def anchor(self) -> np.ndarray:
    """
    Identity anchor A_i = (sum alpha_k * c_k) / (sum alpha_k)
    Returns weighted mean of component values.
    """
    total_w = sum(c.weight for c in self.components)
    if total_w == 0.0:
      raise ValueError("Zero total weight in anchor computation.")
    return sum(c.weight * c.value for c in self.components) /
        total_w

  def fuzz_radius(self, proj_indices=(0, 1, 2)) -> float:
    """
    Fuzz radius at 3D spatial projection.
    proj_indices: which component channels correspond to x,y,z.
    """
    r_obs = self.anchor()[list(proj_indices)]
    total_w = sum(c.weight for c in self.components)
    sq_sum = sum(
      c.weight * np.sum((c.value[list(proj_indices)] - r_obs)**2)
      for c in self.components
    )
    return np.sqrt(sq_sum / total_w)

# --- Projection operator -----------------------------------------------

def project_EM(bundle: IdentityBundle, L_EM: float) -> np.ndarray:
  """
  P_EM: retain only components at scales with L_m <= L_EM.
  Returns the projected anchor value.
  """
  em_components = [
    c for c in bundle.components
    if SCALE_LADDER[c.scale_n].L_m <= L_EM
```

```
  ]
  if not em_components:
    return np.zeros_like(bundle.components[0].value)
  total_w = sum(c.weight for c in em_components)
  return sum(c.weight * c.value for c in em_components) / total_w

# --- Dark density residual ---------------------------------------------

def dark_density_residual(
  rho_full: np.ndarray,   # shape (Nw,)  density as function of w
  w_grid:   np.ndarray,   # shape (Nw,)  w coordinate values
  K_G:      np.ndarray,   # shape (Nw,)  gravitational kernel
  K_EM:     np.ndarray,   # shape (Nw,)  EM kernel
) -> float:
  """
  rho_dark = integral rho(w) [K_G(w) - K_EM(w)] dw
  Numerical trapezoidal integration.
  """
  integrand = rho_full * (K_G - K_EM)
  return float(np.trapz(integrand, w_grid))

# --- Cancellation and hidden intensity ---------------------------------

def observable_channel(c_j: np.ndarray, alpha: np.ndarray) -> float:
  """I_j = alpha^T c_j"""
  return float(alpha @ c_j)

def hidden_intensity(c_j: np.ndarray, W: np.ndarray) -> float:
  """H_j = c_j^T W c_j"""
  return float(c_j @ W @ c_j)

def is_hidden_active(
  c_j: np.ndarray,
  alpha: np.ndarray,
  W: np.ndarray,
  tol: float = 1e-12
) -> bool:
  """Returns True when I_j == 0 but H_j > 0."""
  I = abs(observable_channel(c_j, alpha))
  H = hidden_intensity(c_j, W)
  return (I < tol) and (H > tol)

# --- Effective proper time ---------------------------------------------

def dtau_eff(
  dt:     float,
  v:      float,
  c:      float = 2.998e8,
  Phi_G:  float = 0.0,
  chi_w:  float = 0.0,
) -> float:
  """
  d tau_eff = dt * sqrt(1 - v^2/c^2 + 2*Phi_G/c^2 + chi_w)
  chi_w: dark-scale metric correction (default 0 -> standard GR).
  """
```

```
  arg = 1.0 - (v/c)**2 + 2.0*Phi_G/c**2 + chi_w
  if arg < 0.0:
    raise ValueError(f"Negative radicand in dtau_eff: {arg:.4e}")
  return dt * np.sqrt(arg)

# --- Relation particle
    ------------------------------------------------------

@dataclass
class RelationParticle:
  """
  R_AB = [0, 0, 0, 0 | T_AB]
  Spacetime displacement is zero. Relation vector T carries the
     content.
  """
  T: np.ndarray  # shape (m,), values in {-1, 0, +1}

  @property
  def spacetime_displacement(self) -> np.ndarray:
    return np.zeros(4)

  def measure_A(self, I_A: IdentityBundle, projection_A) -> np.
     ndarray:
    return projection_A(I_A, self)

  def measure_B(self, I_B: IdentityBundle, projection_B) -> np.
     ndarray:
    return projection_B(I_B, self)
```

Listing 1: Identity bundle and projection

---

§9 Code: VSIM Kernel Hook Sketch (C++)

Import block I *standalone, C++23*

The following sketch shows how the theoretical objects above would be wired into the VSIM kernel event system (see WO-64A / WO-64B).

```
// ikk_kernel_hook.hpp  --  WO-64 theoretical integration sketch
// Compile as part of include/core/
#pragma once

#include <array>
#include <cmath>
#include <numeric>
#include <span>
#include <stdexcept>
#include <vector>

namespace ikk {

// ---- Scale entry
    ------------------------------------------------------------

struct ScaleEntry {
```

```
  int          n;
  const char* label;
  const char* dof;
  const char* mediator;
  double       L_m;      // observational limit (m)
};

inline constexpr std::array<ScaleEntry, 7> kScaleLadder = {{
  {0, "S0", "existence",  "---",      1.616e-35},
  {1, "S1", "vector",     "---",      1.0e-19  },
  {2, "S2", "angular",    "gluon",    1.0e-15  },
  {3, "S3", "spatial",    "photon",   1.0e-10  },
  {4, "S4", "time",       "W/Z",      1.0e-18  },
  {5, "S5", "curvature",  "graviton",1.0e+26  },
  {6, "SD", "dark",       "X_D",      1.0e+99  },
}};

// ---- Component ------------------------------------------------------------

struct Component {
  int                 scale_n;
  std::vector<double> value;    // multi-channel value
  double              weight = 1.0;
};

// ---- Identity anchor ------------------------------------------------------

std::vector<double> identity_anchor(
  std::span<const Component> components)
{
  if (components.empty())
    throw std::runtime_error("ikk: empty component set");

  const std::size_t m = components[0].value.size();
  std::vector<double> acc(m, 0.0);
  double total_w = 0.0;

  for (const auto& c : components) {
    total_w += c.weight;
    for (std::size_t k = 0; k < m; ++k)
      acc[k] += c.weight * c.value[k];
  }
  for (auto& v : acc) v /= total_w;
  return acc;
}

// ---- Fuzz radius ----------------------------------------------------------

double fuzz_radius(
  std::span<const Component> components,
  std::span<const std::size_t> proj_idx)   // which channels = 3D
{
  const auto anchor = identity_anchor(components);
  double sq_sum = 0.0, total_w = 0.0;

```

```
  for (const auto& c : components) {
    total_w += c.weight;
    double d2 = 0.0;
    for (auto idx : proj_idx) {
      double diff = c.value[idx] - anchor[idx];
      d2 += diff * diff;
    }
    sq_sum += c.weight * d2;
  }
  return std::sqrt(sq_sum / total_w);
}

// ---- Observable channel and hidden intensity
    ---------------------------------

double observable_channel(
  std::span<const double> c_j,
  std::span<const double> alpha)
{
  return std::inner_product(c_j.begin(), c_j.end(),
                alpha.begin(), 0.0);
}

// W is m x m, row-major
double hidden_intensity(
  std::span<const double> c_j,
  std::span<const double> W_flat,
  std::size_t m)
{
  double result = 0.0;
  for (std::size_t i = 0; i < m; ++i)
    for (std::size_t j = 0; j < m; ++j)
      result += c_j[i] * W_flat[i * m + j] * c_j[j];
  return result;
}

bool is_hidden_active(
  std::span<const double> c_j,
  std::span<const double> alpha,
  std::span<const double> W_flat,
  std::size_t m,
  double tol = 1.0e-12)
{
  const double I = std::abs(observable_channel(c_j, alpha));
  const double H = hidden_intensity(c_j, W_flat, m);
  return (I < tol) && (H > tol);
}

// ---- Effective proper time
    ------------------------------------------------

double dtau_eff(
  double dt,
  double v,
  double Phi_G  = 0.0,
  double chi_w  = 0.0,
  double c      = 2.99792458e8)
{
```

```cpp
  const double arg = 1.0 - (v * v) / (c * c)
           + 2.0 * Phi_G / (c * c)
           + chi_w;
  if (arg < 0.0)
    throw std::runtime_error("ikk::dtau_eff: negative radicand");
  return dt * std::sqrt(arg);
}

// ---- Dark density residual
    ------------------------------------------------

double dark_density_residual(
  std::span<const double> rho_w,    // density as function of w
  std::span<const double> w_grid,   // w coordinate values
  std::span<const double> K_G,      // gravitational kernel
  std::span<const double> K_EM)     // EM kernel
{
  const std::size_t N = rho_w.size();
  std::vector<double> integrand(N);
  for (std::size_t i = 0; i < N; ++i)
    integrand[i] = rho_w[i] * (K_G[i] - K_EM[i]);

  // Trapezoidal rule
  double result = 0.0;
  for (std::size_t i = 0; i + 1 < N; ++i)
    result += 0.5 * (integrand[i] + integrand[i+1])
            * (w_grid[i+1] - w_grid[i]);
  return result;
}

}  // namespace ikk
```

Listing 2: Identity anchor and hidden intensity in VSIM kernel style

---

§10 Open Variable Register

Import block J *living document — update each pass*

Variable	Introduced	Status / constraint needed
$\chi_w(w)$	Eq. (F.5)	Dark-scale metric correction. Formally undefined. Needs physical derivation or phenomenological ansatz from clock-discrepancy data.
$K_G(w)$	Eq. (E.3)	Gravitational projection kernel over dark coordinate. Known property: $\int K_G\, dw = 1$. Explicit form unknown.
$K_{\mathrm{EM}}(w)$	Eq. (E.3)	EM projection kernel. Known property: truncates above L_{EM}. Explicit form unknown.
Π_n	Eq. (B.3)	Packing operator at scale n. Axiom only. Explicit construction at each scale level needed.
$\mathbf{T}_{AB}$	Eq. (G.2)	Trinary relation vector. Space defined. Dynamics not defined.
X_D	Table **??**	Dark mediator. Placeholder only. Mass, spin, coupling unknown.
$\hat{H}_w$	Eq. (extended-TDSE)	Dark-scale Hamiltonian. Without explicit form, the theory has no new quantitative predictions at the QM level.
$\mathbf{W}$	Eq. (D.3)	Hidden-intensity weight matrix. Must be PSD. Specific form for each interaction channel needed.

Table 1: Open variables requiring further formalization.